

INF264 - Homework 2

Credits: Pierre Gillot, Natacha Galmiche, Odin Hoff Gardå

Instructions

- **Format:** Jupyter notebook
- **Deadline:** Thursday 3rd September, 23:59
- **Submission place:** MittUiB

1 Univariate linear regression with gradient descent

In this exercise, we consider the simplest case of linear regression, namely univariate linear regression. Suppose we are given a dataset (x, y) where $x \in \mathbb{R}^N$ represents N samples and $y \in \mathbb{R}^N$ represents the associated target values. Finding the straight line that best fits our dataset (x, y) is equivalent to solving:

$$\min_{(w_0, w_1) \in \mathbb{R}^2} \frac{1}{N} \sum_{n=0}^{N-1} (y_n - (w_0 + w_1 x_n))^2 \iff \min_{w \in \mathbb{R}^2} \frac{1}{N} \|y - \bar{x}w\|_2^2 \quad (1)$$

where $\bar{x} = \begin{bmatrix} \vec{1} | x \end{bmatrix}$ is the vertical concatenation of x with a vector of N ones. Linear problems of the form

$$\min_W \|Y - XW\|_2^2 \quad (2)$$

admit a closed form solution given by

$$(X^T X)^{-1} X^T Y. \quad (3)$$

The matrix $(X^T X)^{-1} X^T$, often denoted X^+ , is called the Moore-Penrose inverse (and sometimes the pseudoinverse) of X . Unfortunately, in many real-world scenarios this inversion problem is too costly and/or unstable to compute directly. In such situations, the gradient descent method is a well-suited solution. In the following questions, you will derive and implement the gradient descent method for the univariate linear regression problem:

1. Derive the gradient of the mean-squared error (MSE) loss $L: \mathbb{R}^2 \rightarrow \mathbb{R}_{\geq 0}$ for the univariate linear regression problem, that is

$$\nabla_w L = \left(\frac{\partial L}{\partial w_0}(w_0, w_1), \frac{\partial L}{\partial w_1}(w_0, w_1) \right) \quad (4)$$

where

$$L(w_0, w_1) = \frac{1}{N} \sum_{n=0}^{N-1} (y_n - (w_0 + w_1 x_n))^2. \quad (5)$$

2. Implement an algorithm which, given a learning rate $\eta > 0$, iteratively updates the weights $w = (w_0, w_1)$ by following the opposite direction of the gradient (also known as the "steepest descent") of the loss at the current estimate. That is, use the update rule

$$w^{t+1} := w^t - \eta \nabla_{w^t} L \quad (6)$$

Hint: You will have to implement a function which for any input $w = (w_0, w_1)$ returns the gradient $\nabla_w L$, based on the equations obtained in the previous question. You can use $w^0 = (0, 0)$ as a starting point for the iteration.

3. Use your algorithm to learn the dataset contained in the file 'unilinear.csv'. You can set the number of gradient descent iterations to 40 and the learning rate to 0.1. Print the found weights, and make a plot with the line $y = w_0 + w_1x$ together with the datapoints. Does the learned line fit the datapoints?
4. Directly compute the exact closed form solution and compare it to the approximate solution obtained previously. Compute the absolute difference between the exact solution and the weights found by using gradient descent. If you increase the number of iterations used in the gradient descent, how does this affect the difference between the two solutions?
5. Perform the gradient descent estimation of the weights for different values of η (for example: $\eta = 0.0001, 0.001, 0.01, 0.1 \dots$). What do you notice? Why is η referred to as the "learning rate"?

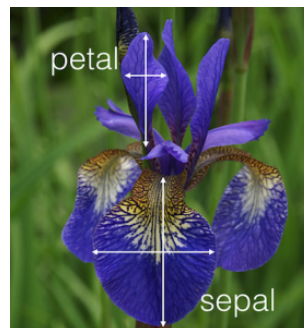
2 Logistic Regression

Logistic Regression (also called Logit Regression) is a generalized linear model which is commonly used to estimate the probability that an instance belongs to a particular class (e.g., what is the probability that a given email is spam?). If the estimated probability is greater than 0.5, then the model predicts that the instance belongs to that class (called the positive class, labeled 1), or else it predicts that it does not (i.e., it belongs to the negative class, labeled 0). This makes it a binary classifier.

We will revisit the Iris dataset to illustrate Logistic Regression. The dataset contains the sepal and petal length and width of 150 iris flowers of three different species: Iris-Setosa, Iris-Versicolor, and Iris-Virginica.



Figure 1: Iris dataset



2.1 Logistic regression on two classes

Complete the following tasks to implement and compare two binary classifiers based on linear regression and logistic regression, respectively.

1. Load the Iris dataset. Since we are doing binary classification, instead of having three labels (for the different species), create two labels 1 and 0, indicating "Iris-Virginica" and "not Iris-Virginica", respectively. We will only use a single feature, namely the one called `petal width (cm)`. Also, you should split the dataset into training, validation and test data (using the ratio 70/15/15, for example).
2. Fit a linear regression model to the feature `petal width (cm)`.
3. Fit a logistic regression model to the same feature.
4. Compare the linear and the logistic regression models by plotting the predicted probability as a function of the `petal width (cm)` for both models. Also include the datapoints in the plot and color them based on their class label.
5. Evaluate both the linear and logistic regression models on your test dataset and compare the accuracy of each model (using a cut-off/threshold value of 0.5 for both models).

2.2 Multi class logistic regression

Your task now is to build a multi-class classifier that can distinguish between the three species Iris-Virginica, Iris-Setosa, and Iris-Versicolor. This task is very similar to the previous one, but now you should use two features and all the labels. Use the features `petal length (cm)` and `petal width (cm)` and include the labels 0, 1 and 2 indicating the three different species of Iris flowers (i.e., keep the original labels from the Iris dataset).

1. Extract the two features you will use together with the corresponding labels, and then split it into training, validation and test data.
2. Fit a multi-class logistic classifier on the training data.
3. Plot the decision boundaries for predicting the labels of the dataset. Feel free to use the code for plotting the boundaries from the previous week's homework. How is it different than the one that was in the previous week's assignment for k -NN?
4. Evaluate the model on the test data and compute the accuracy.

2.3 Tips

1. You can write `x @ y` to compute the matrix product of two NumPy arrays `x` and `y`. To get the transposed of a NumPy matrix/array `x`, simply use `x.T`.
2. Use NumPy's `np.linalg.pinv(x)` to compute the Moore-Penrose inverse (pseudoinverse) of a matrix `x`.
3. You can use `load_iris()` from `sklearn.datasets` to load the Iris dataset. Pass the argument `as_frame=True` to load the dataset as a pandas DataFrame. You will find more details in the documentation here: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html.
4. You can find the documentation for the `LogisticRegression` class here: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.
5. To evaluate the accuracy of the models, you can use the function `accuracy_score()` from `sklearn.metrics`.

2.4 Bonus questions

1. There is also a function called `balanced_accuracy_score()` in `sklearn.metrics`. What is the difference between this function and `accuracy_score()`? When do you think you should pick one over the other?
2. What does the parameters `penalty`, `C` and `solver` of `LogisticRegression` do?